```java
@Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.READ_COMMITTED,
        timeout = 5,
        readOnly = false,
        rollbackFor = IOException.class
)
```

```java
@Transactional
public void createOrder() {

}
```

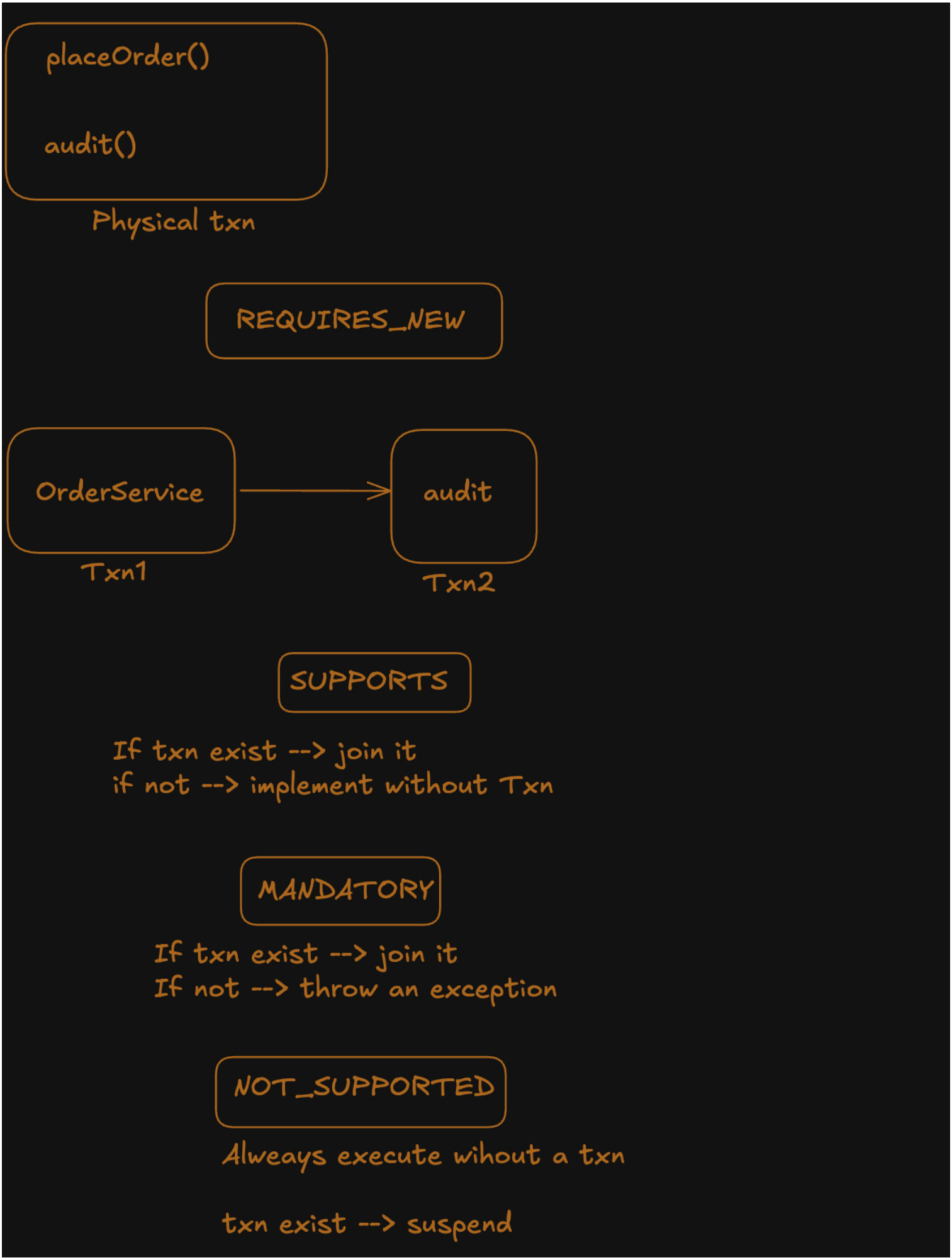
Exceptions :

1. RUntime Exceptions
2. Checked Exceptions

Propagation :

Required

If txn exist --> use it
else --> create new


placeOrder()
audit()
Physical txn
REQUIRES_NEW
OrderService
audit
Txn1
Txn2
SUPPORTS
If txn exist --> join it
if not --> implement without Txn
MANDATORY
If txn exist --> join it
If not --> throw an exception
NOT_SUPPORTED
Alweays execute wihout a txn
txn exist --> suspend

NEVER

I want to execute wihtout txn

txn exist --> throw exception

NESTED

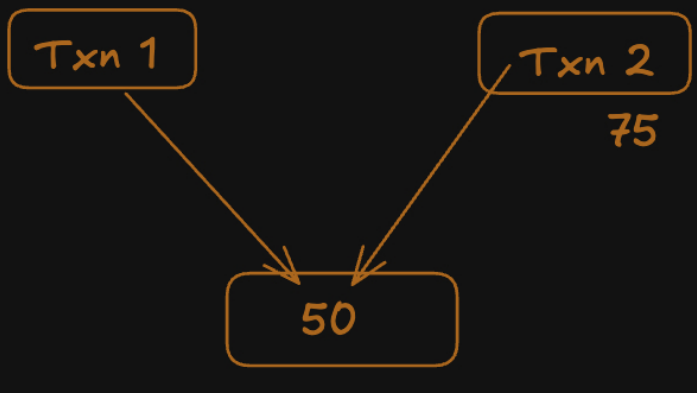
txn exidt -> join
not jexist-> create new

save points

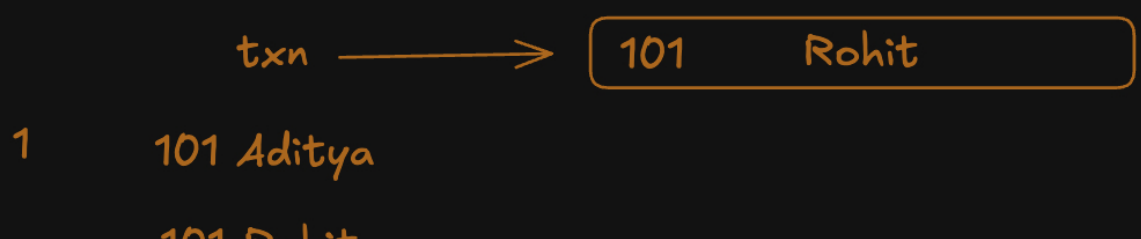
BEGIN
op1
op2
save point
op3
op4
ROLL_BACK

Isolation Levels :

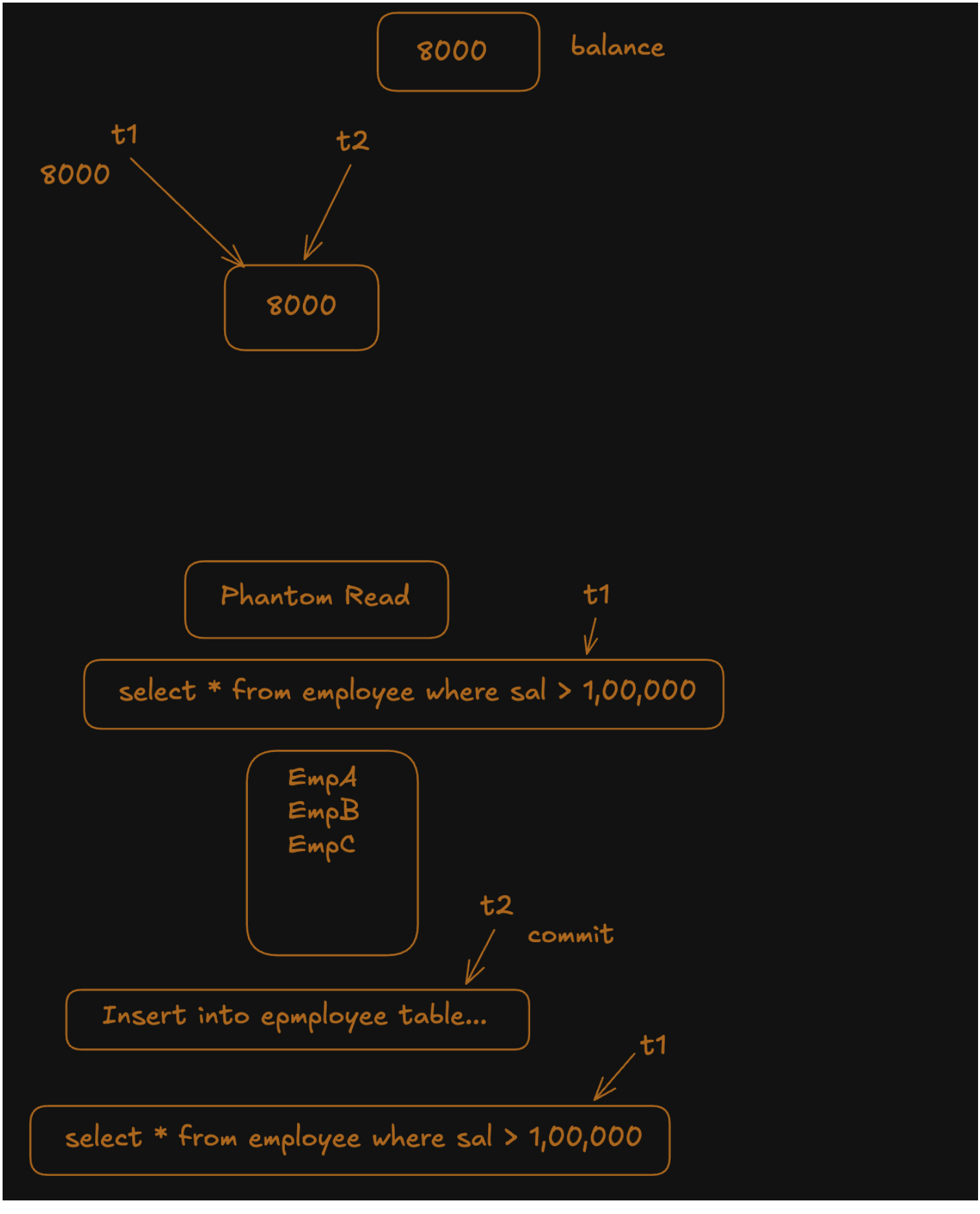
Dirty Read

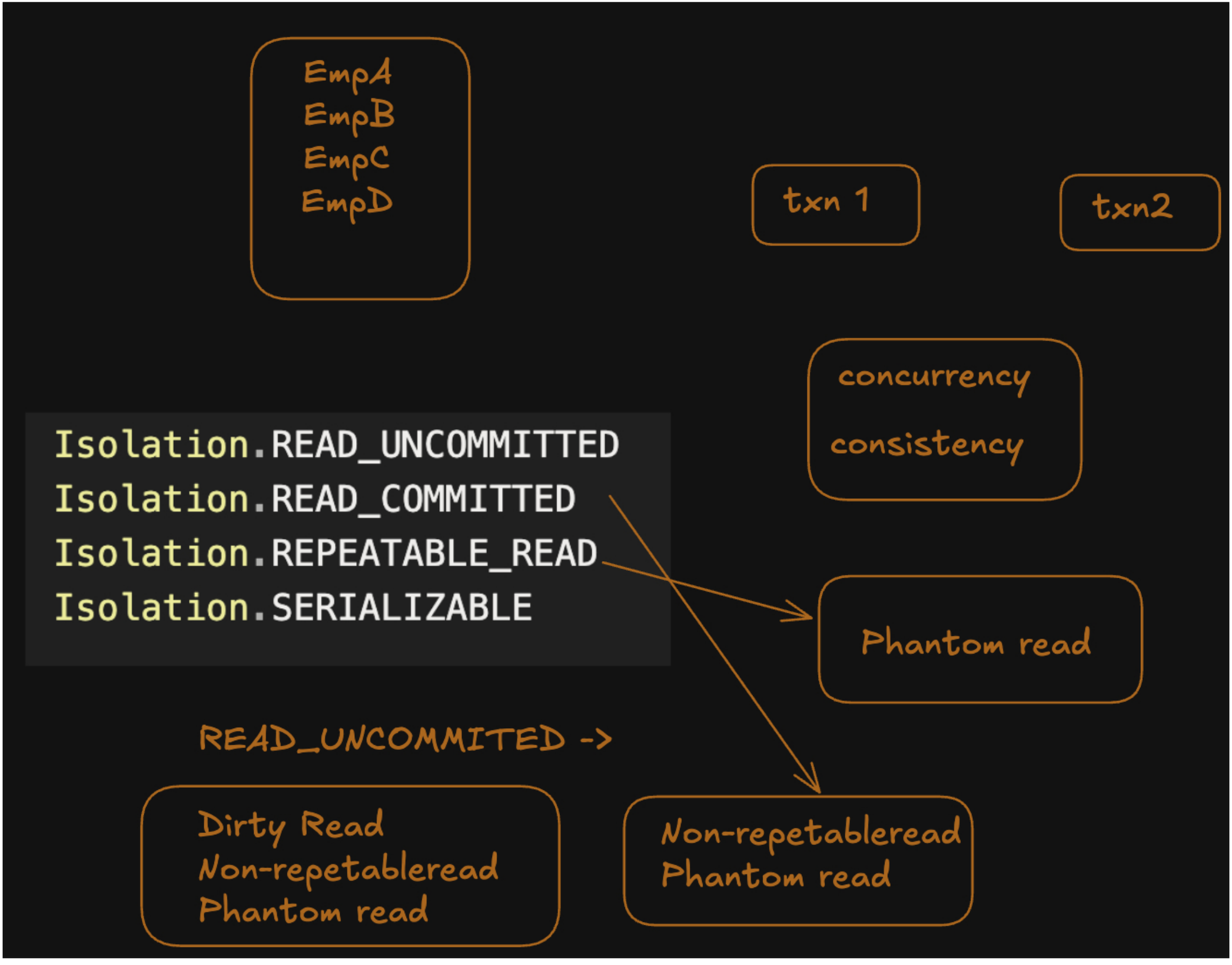


Time	Transaction T1	Transaction T2
1	Begins	
2	Updates balance to ₹2,000	
3		Begins
4		Reads balance as ₹2,000
5	Rolls back	
6	Balance returns to ₹10,000	Continues using ₹2,000



Time	Transaction T1	Transaction T2
1	Begins	
2	Reads balance = ₹10,000	
3		Begins
4		Updates balance to ₹8,000
5		Commits
6	Reads the same row again	
7	Receives balance = ₹8,000	


8000
balance
t1
8000
t2
8000
Phantom Read
t1
select * from employee where sal > 1,00,000
EmpA
EmpB
EmpC
t2
commit
Insert into epmployee table...
t1
select * from employee where sal > 1,00,000


EmpA
EmpB
EmpC
EmpD
txn 1
txn2
concurrency
consistency
Isolation.READ_UNCOMMITTED
Isolation.READ_COMMITTED
Isolation.REPEATABLE_READ
Isolation.SERIALIZABLE
Phantom read
READ_UNCOMMITED ->
Dirty Read
Non-repetableread
Phantom read
Non-repetableread
Phantom read